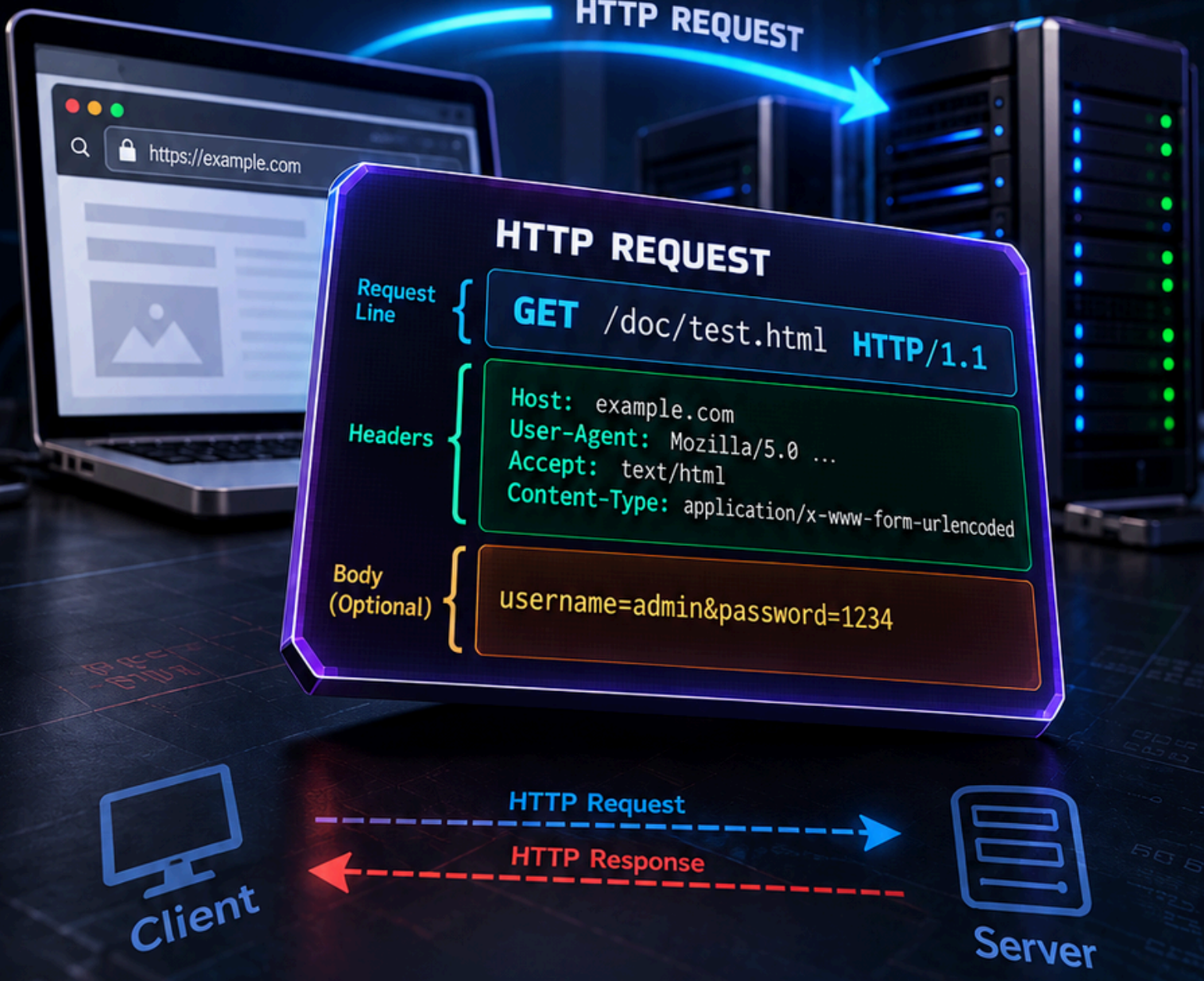
FOUNDATIONS OF WEB APP PENTESTING

1.4 ANATOMY OF AN HTTP REQUEST



HAROON SHAHID

FOUNDATIONS OF WEB APP PENTESTING

1.4 ANATOMY OF AN HTTP REQUEST

HTTP (HyperText Transfer Protocol)

HTTP (Hypertext Transfer Protocol) is an application-layer protocol that defines how clients (such as web browsers or applications) communicate with servers to request resources, submit data, and receive responses. It typically operates over a network and follows a request–response model. HTTP is stateless, meaning the protocol does not inherently maintain a user's session between requests. However, web applications can maintain state using mechanisms such as cookies and session identifiers.

HTTP vs. HTTPS

HTTP defines how clients and servers exchange requests and responses.

HTTPS is HTTP communication protected using TLS (Transport Layer Security).

TLS provides confidentiality, integrity, and server authentication for communication between TLS endpoints.

- **http://** → HTTP without TLS protection.
- **https://** → HTTP protected by TLS.

HTTPS is not a separate HTTP method or protocol version. HTTP/1.1, HTTP/2, and HTTP/3 can all carry HTTP semantics, though HTTP/3 uses QUIC with integrated TLS security.

Security importance: HTTPS protects communication in transit between TLS endpoints. It does not automatically prevent application vulnerabilities such as SQL injection, XSS, CSRF, or broken access control.

HTTP Request

An HTTP request is a structured message sent by a client to a server asking for a specific resource or action. An HTTP request contains a method (such as GET or POST), information identifying the target resource, header fields, and optionally a message body. The exact message structure depends on the HTTP version being used.

HTTP Response

An HTTP response is the structured message sent by the server back to the client after receiving a request. An HTTP response contains a status code (such as 200 or 404), header fields that provide information about the response, and an optional message body containing the requested content or processing result. Some responses, such as responses to HEAD requests and responses with status codes 204 or 304, do not contain response content.

Why It Is Called HTTP (HyperText Transfer Protocol)

It is called HyperText Transfer Protocol because of what it does:

- **HyperText** → Originally, the web was made of HTML pages (HyperText Markup Language). These pages contain clickable links that connect to other pages. That linked text is called hypertext.
- **Transfer** → It moves (transfers) that hypertext and other web data from a server to your browser.
- **Protocol** → A protocol is simply a set of rules. HTTP defines the rules for how a browser asks for something and how a server responds.

So the name literally describes its job:

A rule system used to transfer linked web content between computers.

WHAT IS AN HTTP REQUEST?

An HTTP request is a message sent from the client (browser) to the server.

Purpose:

- Ask for data
- Send data
- Perform an action

Example:

- Open a page
- Submit login form
- Delete something

STRUCTURE OF AN HTTP REQUEST

An HTTP/1.1 request consists of a request line, a header section, an empty line separating the headers from the body, and an optional message body.

1. Request Line: Specifies the HTTP method, request target, and HTTP version.

2. Headers: Provide additional information about the request.
3. Blank Line: Marks the end of the header section.
4. Body (Optional): Contains data sent to the server.

HTTP/2 and HTTP/3 use different message framing mechanisms and do not contain a textual request line in this format.

A. THE REQUEST LINE (FIRST LINE)

In HTTP/1.1, the request line is the first line of the request. It identifies the HTTP method, request target, and HTTP version.

Format:

```
METHOD REQUEST_TARGET HTTP_VERSION
```

Example:

```
GET /doc/test.html HTTP/1.1
```

I. METHOD (HTTP VERB)

HTTP methods are like **instructions you send to a server** telling it what to do with data.

Meaning

- Tells the server what action to perform

Think of it as:

- "What do you want to do?"

Common Methods

GET

- Requests a representation of a resource from the server.
- GET is defined as a safe method, meaning the client does not request a state-changing operation.
- However, GET requests may still produce incidental side effects, such as logging or analytics. Vulnerable or improperly designed applications may also perform sensitive state-changing actions through GET requests.
- Security importance: State-changing GET endpoints can introduce CSRF vulnerabilities because browsers may automatically initiate GET requests through links, navigation, and other mechanisms.

POST

- Sends data to the server for processing according to the functionality of the target resource.
- Common uses include:
    - Submitting login forms.
    - Registering users.
    - Creating resources.
    - Updating information.
    - Performing application-specific actions.
- Unlike GET, POST is not classified as a safe or inherently idempotent method.

PUT

- Requests creation or replacement of the state of the target resource using the supplied representation. A successful PUT can create a resource if it does not already exist and the server permits it.

PATCH

- Requests partial modification of a resource by applying changes described in the request. It does not necessarily replace the entire resource.

DELETE

- Requests removal of the association between the target URI and its current resource functionality. It does not necessarily mean that a physical file or database record is permanently deleted.

HEAD

- Works similarly to GET, except that the server must not include response content in the HEAD response.
- It is used to retrieve information about a resource, such as response headers, without downloading the resource's content.
- The response headers should generally match those of an equivalent GET response, although some headers may differ or be omitted.

OPTIONS

- Requests information about the communication options supported by a target resource or server.
- It can be used to discover supported HTTP methods or other capabilities, although the server may not provide a complete method list.
- Browsers also use OPTIONS requests for certain CORS preflight checks.

TRACE

- Requests a diagnostic loopback of the HTTP request message.

- When supported, the final recipient should return a representation of the received request in the response, excluding fields that may expose sensitive information.
- It is primarily used for debugging and examining how requests are received.
- A client must not send a request body with TRACE.
- TRACE is commonly disabled because reflecting sensitive request information can introduce security risks when implementations handle it improperly.

CONNECT

- Requests that an HTTP proxy establish a tunnel to a specified destination.
- It is commonly used when accessing HTTPS websites through an HTTP proxy.
- CONNECT does not itself encrypt communication. HTTPS security comes from TLS, which can operate through the established tunnel.

Safe Vs. Idempotent HTTP Methods

HTTP methods have two important properties: safety and idempotency.

Safe Methods

A method is considered safe when its defined semantics are essentially read-only. The client does not request a state-changing operation.

Safe methods defined by HTTP include:

- GET
- HEAD
- OPTIONS
- TRACE

A safe method may still cause incidental server-side effects, such as logging.

Idempotent Methods

A method is idempotent if performing the same request multiple times has the same intended effect on server state as performing it once.

For example, consider:

```
PUT /users/123 HTTP/1.1
Host: example.com
Content-Type: application/json
Content-Length: 18

{"username":"Ali"}
```

Sending this identical PUT request multiple times should have the same intended effect on the target resource as sending it once.

Similarly, repeatedly deleting the same resource does not necessarily produce identical HTTP responses, but the intended result can remain the same: the resource is no longer associated with the target URI.

Important: Idempotency concerns the intended effect, not necessarily identical response status codes, logs, or other incidental side effects.

Method	Safe	Idempotent by defined semantics
GET	Yes	Yes
HEAD	Yes	Yes
OPTIONS	Yes	Yes
TRACE	Yes	Yes
PUT	No	Yes
DELETE	No	Yes
POST	No	Not inherently
PATCH	No	Not inherently

An application may implement a particular POST or PATCH operation idempotently, but the HTTP method itself does not guarantee that property.

Security Importance: Understanding safety and idempotency helps when testing state-changing endpoints, retry behavior, CSRF, and unexpected business-logic operations.

II. REQUEST TARGET, PATH & QUERY STRING

Meaning

- Identifies the resource or endpoint being requested.

Example:

- **/doc/test.html**

The application uses this path to determine which resource or functionality the client is requesting.

In traditional static websites, the path may correspond to an actual file or directory on the server.

However, in modern web applications, paths are often mapped to application routes, controller functions, database queries, or API endpoints rather than physical files.

Important distinction: Path vs. Request Target

Consider the following HTTP/1.1 request:

```
GET /products?id=123&category=books HTTP/1.1
Host: example.com
```

Here:

- **Path:** `/products`
- **Query String:** `id=123&category=books`
- **Request Target:** `/products?id=123&category=books`

In normal requests to an origin server, the request target usually contains the path and optional query string.

However, other request-target formats exist, including absolute URLs in certain proxy requests, host-and-port targets for CONNECT, and `*` for server-wide OPTIONS requests.

Important: A URL fragment such as `#section1` is not sent as part of the HTTP request target.

Query Parameters Vs. Request Body

Query parameters are not exclusive to GET requests.

Both GET and POST requests can contain query parameters in their request targets.

For example:

```
POST /update-profile?user_id=123 HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 11

name=Haroon
```

In this example:

Query parameter: `user_id=123`

Request body parameter: `name=Haroon`

The application receives information in two different locations.

Important: HTTP itself does not universally define how an application should merge or prioritize query parameters and body parameters.

Frameworks may expose them separately or combine them through framework-specific mechanisms.

Security Importance: During penetration testing, inspect both locations. Parameter conflicts, unexpected parameter precedence, or inconsistent validation may contribute to application vulnerabilities.

This is a particularly important distinction when you begin testing parameter pollution and authorization checks.

URL Encoding And Percent-Encoding

URLs and request targets can contain percent-encoded representations of bytes.

Percent-encoding uses the format: **%HH**

where **HH** represents a hexadecimal byte value.

Examples:

Character	Percent-encoded form
Space	**%20**
/	**%2F**
?	**%3F**
&	**%26**
=	**%3D**

Important: Encoding changes how characters or bytes are represented. It does not guarantee that encoded characters will be treated as literal characters by every server or application.

For example: **/admin%2Fpanel**

may be interpreted differently depending on how the server, proxy, framework, or router handles encoded slashes.

In **application/x-www-form-urlencoded** data, a plus sign (**+**) represents a space, while a literal plus sign can be represented as **%2B**.

Security Importance: Different decoding and normalization behaviors can matter when testing path traversal, access-control bypasses, routing inconsistencies, and input validation.

III. HTTP VERSION

Example

HTTP/1.1

Meaning

Version of HTTP protocol used

Important points:

Version	Description
HTTP/1.1	Uses a text-based start line and headers. Commonly operates over TCP. Message bodies can contain text or binary data.
HTTP/2	Uses binary framing, multiplexing, and header compression (HPACK). Multiple request and response streams can share one connection.
HTTP/3	Uses QUIC, which operates over UDP and incorporates TLS 1.3 security for HTTPS connections. It provides multiplexed streams without TCP-level head-of-line blocking between them.

All three versions preserve essentially the same HTTP method and response semantics, although their message framing differs.

Important for Burp Suite: A request displayed in a readable editor does not necessarily mean the underlying connection uses HTTP/1.1. Burp can present decoded HTTP/2 requests in an understandable format.

HTTP/2 uses pseudo-headers such as:

> **:method: GET**
> **:scheme: https**
> **:authority: example.com**
> **:path: /products?id=123**

Important: HTTP/2 pseudo-header fields are special control fields, not ordinary HTTP header fields. They convey information such as the method, scheme, authority, and path. They must appear before regular header fields within the HTTP/2 header block.

When **:authority** is present in an HTTP/2 request, the recipient must use it instead of the **Host** header to determine the target URI. Clients must not generate requests containing conflicting **Host** and **:authority** values. Servers should treat requests containing mismatched values as malformed.

HTTP/2 and HTTP/3 do not include a literal **HTTP/2** or **HTTP/3** request-line version string in each request.

These are the readable representations of HTTP/2 pseudo-header fields. HTTP/2 transmits them using binary framing and header compression, rather than as a literal HTTP/1.1-style request line.

These represent information that appears in different parts of an HTTP/1.1 request.

Note:

- Version itself is not always a vulnerability
- But sometimes affects advanced attacks

B. HEADERS (METADATA)

Headers are Metadata. Metadata in simple words is data about data. For example, an image is data, but when the image was taken, what is the size of the image, dimensions etc, all these information are also data about that image.

Headers = “extra information about the request”

Headers provide additional information about the request, such as:

- The target hostname.
- The client's supported media types and encodings.
- Authentication credentials, when applicable.
- The request body's content type and length.
- Other information used to process the request.

Important: Request headers do not necessarily prove the client's identity. Many header values can be modified or spoofed.

CASE SENSITIVITY

HTTP method names and HTTP header names have different case-sensitivity rules.

HTTP Methods

HTTP method names are case-sensitive.

For example:

`GET` and `get`

are not necessarily the same method.

Standardized HTTP methods conventionally use uppercase letters.

HTTP Header Names

HTTP header field names are case-insensitive.

For example, the following names are equivalent under HTTP semantics:

```
Content-Type: application/json
content-type: application/json
```

CONTENT-TYPE: application/json

Important: This case-insensitivity applies to header field names, not automatically to every header value.

Different header values can have their own case-sensitivity rules.

In HTTP/2 and HTTP/3, regular header field names must be lowercase when encoded in their respective field blocks.

Security Importance: Differences in how proxies, servers, and applications handle HTTP syntax can sometimes contribute to parsing inconsistencies.

I. HOST

Example:

Host: www.test101.com

Meaning:

Specifies the hostname and optional port of the target HTTP resource.

The server or reverse proxy may use this header to determine which website or application should handle the request.

In HTTP/1.1, clients must include a Host header.

Important: The Host header does not by itself determine the actual network connection destination. The connection destination and, for HTTPS, TLS information such as SNI can also matter.

Security Importance:

Improper validation or trust of the Host header can introduce vulnerabilities such as:

- Host header injection.
- Password-reset poisoning.
- Web cache poisoning.

Pentesting Scope:

Always test only the hosts, applications, and assets authorized by the engagement's scope.

II. USER-AGENT

Example:

User-Agent: Mozilla/4.0

Meaning:

Identifies or describes the client software making the request, such as the browser and sometimes its operating system.

May Include:

- Browser name.
- Operating system.
- Browser or client version information.

Important:

User-Agent information is self-reported and can be modified or spoofed. It should not be trusted as proof of the user's identity or actual environment.

Security Insight:

If an application stores attacker-controlled User-Agent values and later displays them in a web interface without appropriate context-sensitive output encoding or sanitization, it may introduce stored XSS.

Simply storing or logging a malicious User-Agent value does not automatically create an XSS vulnerability.

Example XSS Test String:

<script>alert(1)</script>

Important: This test string only demonstrates script execution when it reaches a browser context that interprets it as executable HTML/JavaScript. It will not execute merely because it appears in a server log or as plain text.

III. ACCEPT

- Indicates which media types the client is willing to receive in the response.
- Example:
  - **Accept: image/gif, image/jpeg, */***
  - **image/gif** → GIF images.
  - **image/jpeg** → JPEG images.
  - ***/*** → Any media type.
- The server can use this information when selecting a suitable response representation.
- **Accept** describes acceptable response media types.
- **Content-Type** describes the media type of the content being sent in a particular message.
- For example:
  - **Accept: application/json**

    - **Content-Type: application/json**
- The first means the client accepts JSON responses.
- The second means the request body is represented as JSON.

IV. ACCEPT-LANGUAGE

Example:

Accept-Language: en-us

Meaning:

- Preferred language

Example:

- en-us → English

Server may respond in that language if available

V. ACCEPT-ENCODING

- Specifies the content encodings the client supports for responses.
- Example: **Accept-Encoding: gzip, deflate, br**
- **gzip** → gzip compression.
- **deflate** → deflate compression.
- **br** → Brotli compression.
- The server may choose a supported encoding to reduce the amount of data transferred.
- The response indicates the selected encoding using the **Content-Encoding** header.
- The browser normally decompresses supported encoded content automatically.

VI. COOKIE

Example:

Cookie: session_id=abc123

Meaning:

Sends applicable cookies previously stored by the browser to the server.

Purpose:

Cookies may be used for:

- Session tracking.
- Authentication.

- User preferences.
- Analytics.
- Other application-specific functionality.

Important:

Not every cookie contains session information, and not every cookie identifies an authenticated user.

Security Risk:

If an attacker obtains a valid authentication session cookie, they may be able to impersonate the authenticated user by reusing it, depending on server-side session validation and additional security controls.

This is known as **Session Hijacking**.

However, stealing an arbitrary cookie does not automatically grant authenticated access.

Cookie vs. Set-Cookie

These are two different HTTP headers.

Set-Cookie

Typically sent by the server in an HTTP response to instruct the browser to store a cookie.

Example

```
Set-Cookie: session_id=abc123; Path=/; Secure; HttpOnly; SameSite=Lax
```

Cookie:

Sent by the browser in subsequent requests when the stored cookie is applicable.

```
Cookie: session_id=abc123
```

This distinction is particularly important for understanding session management, authentication, and CSRF.

VII. CONTENT-LENGTH

- Example: `Content-Length: 35`
- Specifies the message body's length in bytes (octets), when used for HTTP/1.1 message framing.
- It is not restricted to particular HTTP methods.

- A request may use **Content-Length: 0** to indicate a zero-length body.
- HTTP/1.1 also supports message framing through **Transfer-Encoding**, such as **chunked**, instead of **Content-Length**.
- Security importance: Incorrect or conflicting message-length information can create HTTP parsing problems.
- HTTP request smuggling may occur when different HTTP components, such as a reverse proxy and backend server, disagree about where a request ends and the next request begins.
- However, an invalid Content-Length value by itself does not automatically establish request smuggling.
- **Important:** Content-Length measures bytes (octets), not characters. In UTF-8, certain characters occupy multiple bytes, so the number of characters does not always equal the Content-Length value.
- **Important:** In HTTP/1.1, a compliant sender must not send both **Content-Length** and **Transfer-Encoding** in the same message. Such conflicting framing information can cause parsing inconsistencies, especially when intermediaries and backend servers handle the message differently.

VIII. CONTENT-TYPE, AUTHORIZATION, ORIGIN

Header	Purpose
Content-Type	Describes the media type of the request body, such as **application/json** or **application/x-www-form-urlencoded**.
Authorization	Carries credentials used for HTTP authentication, such as a bearer access token.
Origin	Indicates the origin responsible for initiating a request in contexts where the browser includes it. Often relevant to CORS and CSRF defenses.

For example:

```
POST /api/profile HTTP/1.1
Host: example.com
Authorization: Bearer example-token
Origin: https://example.com
Content-Type: application/json
Content-Length: 20

{"display":"Haroon"}
```

This is an illustrative HTTP/1.1 request containing a JSON body of 20 bytes. The Content-Type header identifies the body as JSON, the Authorization header carries an example bearer token, and the Origin header identifies the origin responsible for initiating the request.

IX. OTHER IMPORTANT REQUEST HEADERS

Header	Purpose
Referer	Provides information about the resource from which the request originated, subject to browser referrer policies.
Sec-Fetch-Site	Indicates the relationship between the request initiator and the target, such as **same-origin**, **same-site**, or **cross-site**.
Transfer-Encoding	Specifies transfer codings used for message framing or transfer, such as **chunked** in HTTP/1.1. It is not used as an HTTP/2 or HTTP/3 message-framing mechanism.
X-Forwarded-For	Commonly used by proxies to convey information about client IP addresses. It must not be blindly trusted because untrusted clients may supply or influence it.

These headers can be relevant when investigating:

- CSRF protections.
- Request smuggling.
- Reverse-proxy configurations.
- IP-based access controls.
- Origin validation and browser request behavior.

Important: The presence of a header does not automatically establish that its contents are trustworthy.

C. BLANK LINE

- In HTTP/1.1, an empty line marks the end of the header section.
- If a request body exists, it starts immediately after this empty line.
- Even when there is no body, the empty line still marks the end of the headers.
- HTTP/2 and HTTP/3 use binary framing rather than this textual separator.

IMPORTANT: CRLF IN HTTP/1.1

HTTP/1.1 uses CRLF sequences as standard line terminators.

CRLF stands for:

CR → Carriage Return (**\r**)

LF → Line Feed (**\n**)

Together: **\r\n**

Each HTTP/1.1 request-line and header field line is terminated by CRLF.

The end of the header section is represented by an additional CRLF, producing: **\r\n\r\n**

after the final header's text.

For example, the following is a readable representation of an HTTP/1.1 request with escaped line endings:

```
POST /login HTTP/1.1\r\n
Host: example.com\r\n
Content-Length: 4\r\n
\r\n
test
```

The **\r\n** sequences above represent actual CRLF bytes when transmitted, not literal backslash characters.

Security Importance: HTTP message parsing differences involving line endings, whitespace, and message framing can contribute to request smuggling vulnerabilities.

Or to clarify it:

```
POST /login HTTP/1.1\r\nHost: example.com\r\nContent-Length: 4\r\n\r\ntest
```

This is an escaped representation of the transmitted bytes. Each **\r\n** represents the actual CRLF byte sequence (0D 0A), not four literal characters transmitted over the network.

D. REQUEST BODY

Meaning

The request body is the optional portion of an HTTP request that carries data sent to the server.

Commonly Used With

- POST
- PUT
- PATCH

However, the presence of a body is not universally restricted to these methods.

For example, a GET request can technically contain content under HTTP message framing rules, but GET request content has no generally defined semantics and may be rejected by some implementations.

Types of Data

A request body may contain:

- Form data.
- JSON.
- XML.
- Uploaded files.
- Other application-specific content.

The **Content-Type** header tells the server what media type the request body uses.

Common Request Body Formats

1. application/x-www-form-urlencoded

Commonly used by HTML forms.

Example:

```
Content-Type: application/x-www-form-urlencoded

username=haroon&password=123456
```

2. application/json

Commonly used by APIs.

Example:

```
Content-Type: application/json

{"username":"haroon","password":"123456"}
```

3. multipart/form-data

Commonly used for forms containing files or multiple parts.

Example:

```
Content-Type: multipart/form-data; boundary=abc123

--abc123
Content-Disposition: form-data; name="username"
```

```
haroon
--abc123--
```

The boundary parameter identifies the delimiter separating parts of the multipart body.

4. text/plain

Represents plain text.

Example:

```
Content-Type: text/plain

Hello World
```

Important: The Content-Type field describes the request content. It does not itself guarantee that the content matches the declared format.

Different endpoints may accept, reject, or parse the same request body differently based on Content-Type.

Security Importance: Content-type handling and parser inconsistencies can influence request validation, CSRF exploitability, and application-specific injection vulnerabilities.

Example

An HTML form submission using `application/x-www-form-urlencoded` may contain the following request body:

```
username=haroon&password=123456
```

Used For:

- Login.
- Form submission.
- API requests.
- File uploads.
- Creating or modifying resources.

COMPLETE EXAMPLE

```
GET /doc/test.html HTTP/1.1
Host: www.test101.com
Accept: text/html, */*
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US
User-Agent: Mozilla/5.0
```

This is an example of an HTTP/1.1 GET request without a request body.

The empty line after the headers marks the end of the HTTP/1.1 header section.

The text **(No body in GET)** is not part of the actual HTTP request; it is only an explanatory note.

Although GET requests normally do not contain bodies, HTTP message framing does not categorically make them impossible.

PENTESTER VIEW

HTTP requests contain data that may be controlled or manipulated by a client.

During penetration testing, inspect request targets, query parameters, headers, cookies, and request bodies as potential attack surfaces.

However, not every input is vulnerable, and the ability to modify a request manually does not automatically mean an attacker can reproduce the same modification through a victim's browser.

Server-side applications must validate, authorize, and safely handle untrusted input according to its intended use.

IMPORTANT ATTACK POINTS

Request Line

- Change method
- Change path

Headers

- Modify User-Agent
- Modify Cookie
- Modify Host

Body

- Inject payloads

KEY VULNERABILITIES LINKED

Input or component	Potential security issue
Request path	Path traversal, routing and access-control vulnerabilities

Query parameters	SQL injection, XSS, IDOR, business-logic flaws
User-Agent	Stored XSS if unsafely displayed in an executable context
Host	Host header injection, password-reset poisoning, web cache poisoning
Cookie	Session hijacking if a valid authentication cookie is compromised; cookie tampering if values are improperly trusted
Content-Length / Transfer-Encoding	HTTP request smuggling due to inconsistent message framing
Request body	Injection, mass assignment, parameter tampering, authorization flaws
Authorization header	Authentication and authorization implementation weaknesses

Important: These are potential attack surfaces, not guaranteed vulnerabilities. A vulnerability exists only when the application's implementation contains an exploitable weakness.

IMPORTANT CONCEPTS

I. STATELESSNESS

HTTP is a stateless protocol, meaning each request is independent at the protocol level. HTTP itself does not require a persistent user session to be maintained between requests.

However, web applications can maintain state using sessions, cookies, authentication tokens, and server-side storage.

Therefore, statelessness does not mean that a server is incapable of remembering a user or tracking their activity across requests.

II. SESSION TRACKING

- Although HTTP is stateless, web applications can maintain user sessions across multiple requests.
- A common approach is to create a server-side session and provide the browser with a session identifier stored in a cookie.
- The browser sends the cookie with subsequent applicable requests, allowing the server to identify the corresponding session.
- Applications may also use other mechanisms, such as authorization tokens, to associate requests with authenticated users.

HTTP/1.1 vs HTTP/2

- HTTP/1.1 uses a human-readable textual format for its start line and headers, while HTTP/2 uses binary frames and pseudo-headers.
- HTTP/2 introduces multiplexing and header compression, which can improve communication efficiency.
- Modern interception proxies can decode HTTP/2 and present requests in a readable format.
- Some advanced vulnerabilities, including HTTP/2 request smuggling and downgrade-related parsing issues, depend on how different components implement or translate HTTP versions.

BURP SUITE REPEATER (IMPORTANT TOOL)

What it does:

- Take a request
- Modify it
- Send it again

You can:

- Change method
- Change headers
- Change body

Purpose:

- Test how server reacts

Burp Suite Repeater allows a penetration tester to manually modify and resend HTTP requests, inspect responses, and compare how an application behaves when request parameters, methods, headers, cookies, or bodies change.

However, a request succeeding in Repeater does not necessarily demonstrate that the same request can be generated through a normal browser or attacker-controlled webpage.

FULLY ANNOTATED HTTP/1.1 REQUEST

Consider the following example of an HTTP/1.1 request submitting a login form:

```
POST /login HTTP/1.1
Host: example.com
```

Content-Type: application/x-www-form-urlencoded
Content-Length: 31
Accept: text/html
Cookie: session=abc123

username=haroon&password=123456

Breakdown

Component	Explanation
POST	The HTTP method. It tells the server to process submitted data according to the target endpoint's functionality.
/login	The request target, identifying the login endpoint.
HTTP/1.1	The HTTP protocol version.
Host: example.com	Identifies the target hostname.
Content-Type	Indicates that the request body contains URL-encoded form data.
Content-Length: 31	Specifies that the request body contains 31 bytes.
Accept: text/html	Indicates that the client accepts HTML responses.
Cookie: session=abc123	Sends an applicable cookie to the server.
Blank line	Separates the HTTP/1.1 header section from the request body.
username=haroon&password=123456	The actual form-encoded data sent to the server.

Important: The presence of a session cookie in this example does not necessarily mean the user is already authenticated. A cookie may identify an anonymous session or serve another purpose.

This example brings together the different components of an HTTP/1.1 request in one place.

FINAL CORE UNDERSTANDING

- **HTTP Request:** A structured message sent by a client to request a resource or action from a server.
- **HTTP/1.1 Request Line:** HTTP method + request target + HTTP version.
- **Headers:** Additional information used to describe or process the request.
- **Blank Line (HTTP/1.1):** Marks the end of the header section.

- **Body:** Optional content transmitted with the request, such as form data, JSON, or files.
- **Important:** HTTP/2 and HTTP/3 preserve HTTP's core semantics but use different message framing mechanisms.
